DevOps Pipeline Implementation

Scientific Calculator Application

Roll Number: MT2025043

March 5, 2026

Contents

1 Introduction **2**

2 What and Why of DevOps **3**
2.1 What is DevOps 3
2.2 Why DevOps 3

3 Tools Used **4**

4 Source Code Management using GitHub **5**
4.1 GitHub Repository 5
4.2 Commit History 6
4.3 Git Commands Used 7

5 Continuous Integration using Jenkins **8**
5.1 Jenkins Pipeline Stages 8
5.2 Jenkins Pipeline Execution 9
5.3 Jenkins Console Output 10

6 Containerization using Docker **11**
6.1 Dockerfile 11
6.2 Docker Hub Repository 12

7 Deployment using Ansible **13**
7.1 Ansible Playbook Execution 13
7.2 Ansible Command Used 14

8 Scientific Calculator Application Output **15**
8.1 Application Execution 15

9 Project Links **16**

10 Conclusion **17**

Chapter 1

Introduction

DevOps is a modern software development approach that combines software development and IT operations to shorten the system development life cycle while delivering features, fixes, and updates frequently.

In this project, a DevOps pipeline was implemented for a Scientific Calculator application. The pipeline automates the entire workflow from source code management to deployment using modern DevOps tools.

The pipeline includes:

- GitHub for version control
- Jenkins for Continuous Integration
- Docker for containerization
- Docker Hub for image storage
- Ansible for deployment automation

Chapter 2

What and Why of DevOps

2.1 What is DevOps

DevOps is a combination of development and operations practices that aims to automate and integrate the processes between software development and IT teams. It allows organizations to build, test, and release software faster and more reliably.

2.2 Why DevOps

DevOps provides several advantages:

- Faster software development and deployment
- Continuous integration and delivery
- Improved collaboration between teams
- Automation of manual tasks
- Reduced deployment failures

Chapter 3

Tools Used

The following tools were used in this project:

- **GitHub** – Source Code Management
- **Jenkins** – Continuous Integration
- **Docker** – Containerization
- **Docker Hub** – Container Image Repository
- **Ansible** – Deployment Automation

Chapter 4

Source Code Management using GitHub

GitHub was used to store the project source code and manage version control. Developers push their code to the GitHub repository, which triggers the Jenkins pipeline automatically.

4.1 GitHub Repository

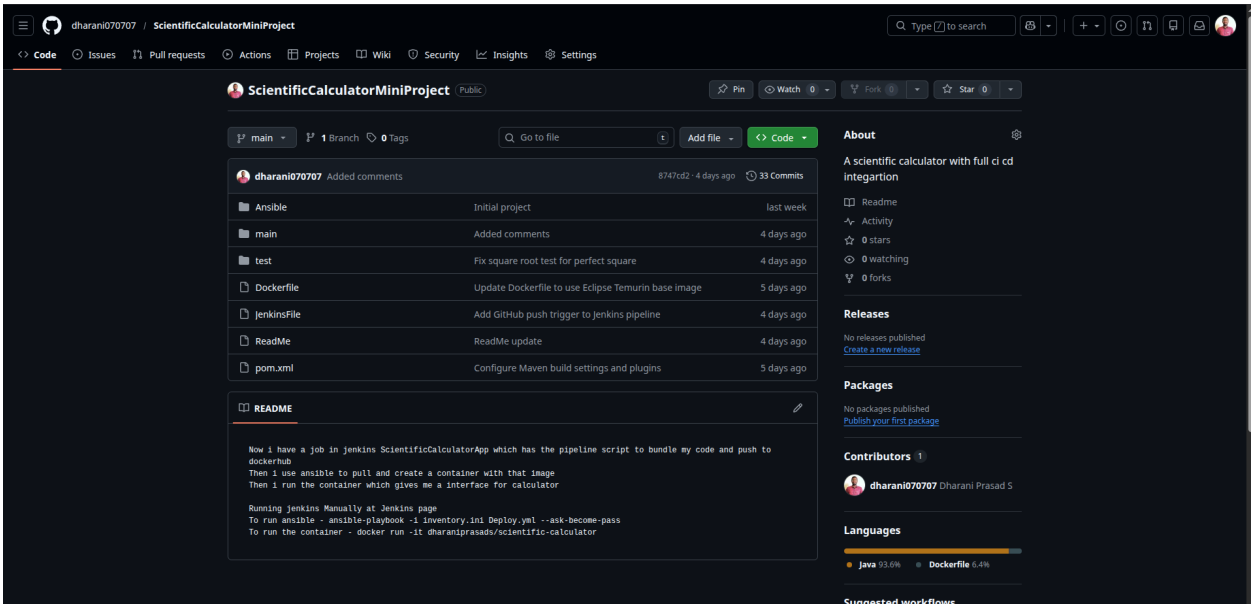


Figure 4.1: GitHub Repository for Scientific Calculator

4.2 Commit History

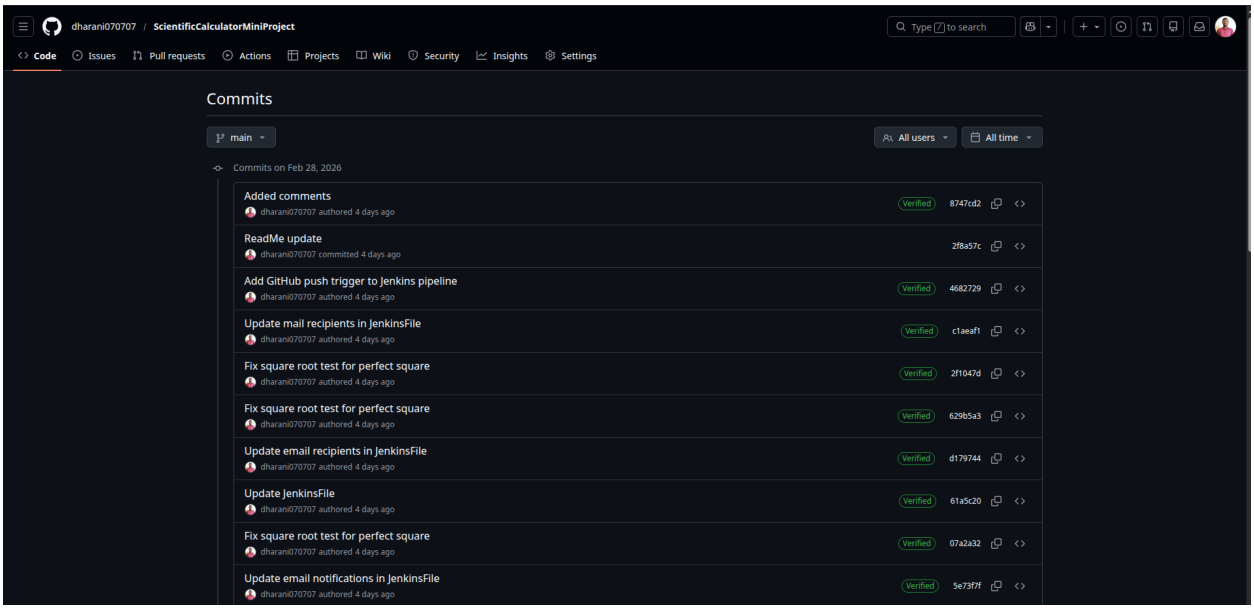


Figure 4.2: GitHub Commit History

4.3 Git Commands Used

```
git init
git add .
git commit -m "Initial Commit"
git branch -M main
git remote add origin
https://github.com/dharani070707/ScientificCalculatorMiniProject.git
git push -u origin main
```

Chapter 5

Continuous Integration using Jenkins

Jenkins is an automation server used to implement continuous integration. Whenever a new commit is pushed to GitHub, Jenkins automatically triggers a pipeline to build and test the application.

5.1 Jenkins Pipeline Stages

The pipeline consists of the following stages:

- Checkout Source Code
- Build Application
- Run Tests
- Package Application
- Build Docker Image
- Push Docker Image to Docker Hub

5.2 Jenkins Pipeline Execution

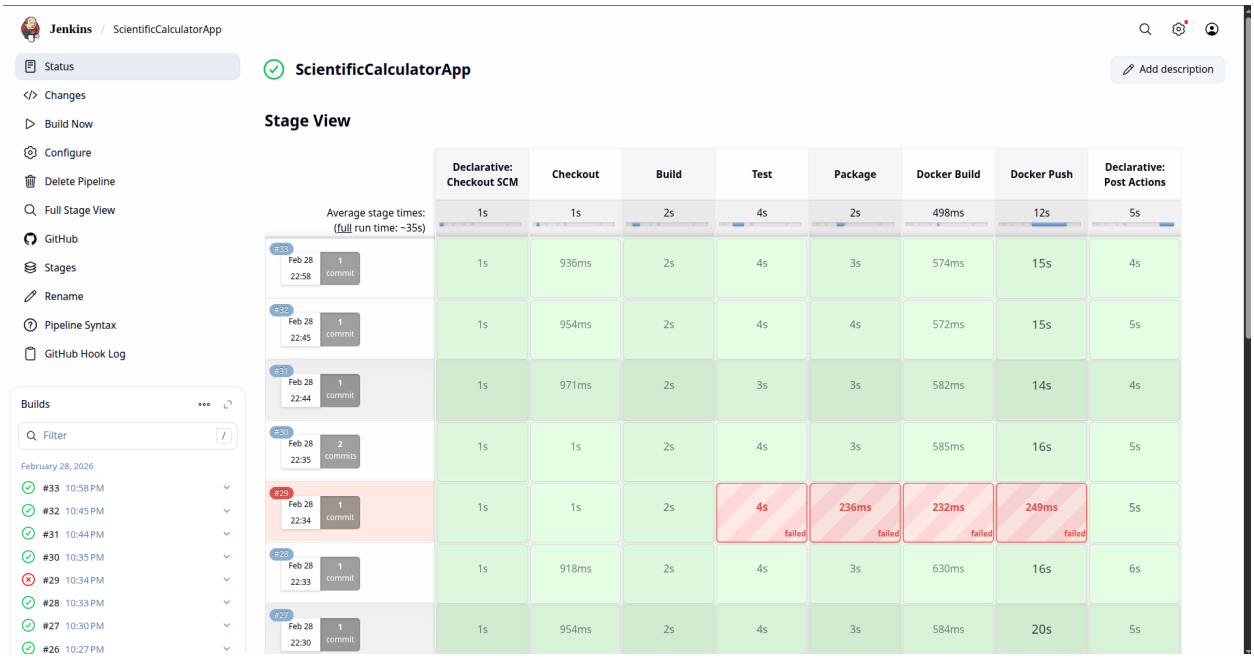


Figure 5.1: Jenkins Pipeline Stage View

5.3 Jenkins Console Output

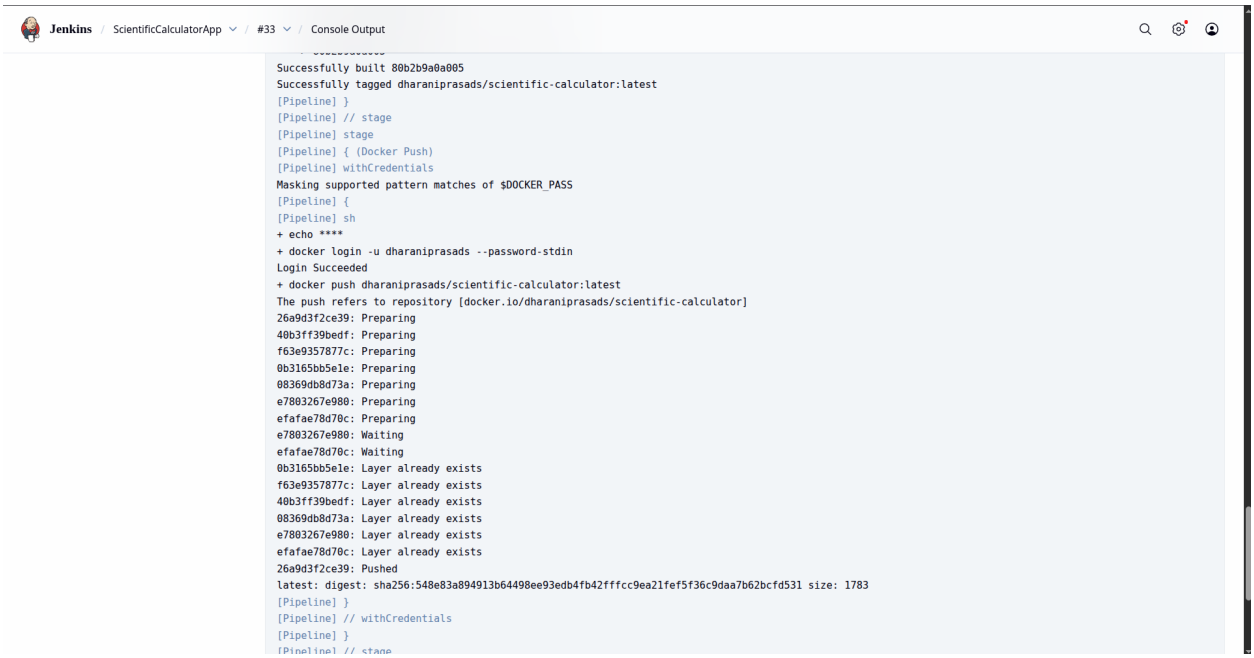


Figure 5.2: Jenkins Console Output for Docker Push

Chapter 6

Containerization using Docker

Docker is used to package the application and its dependencies into a container image. This ensures that the application runs consistently across different environments.

6.1 Dockerfile

```
FROM eclipse-temurin:17

WORKDIR /app

COPY target/app.jar app.jar

ENTRYPOINT ["java","-jar","app.jar"]
```

6.2 Docker Hub Repository

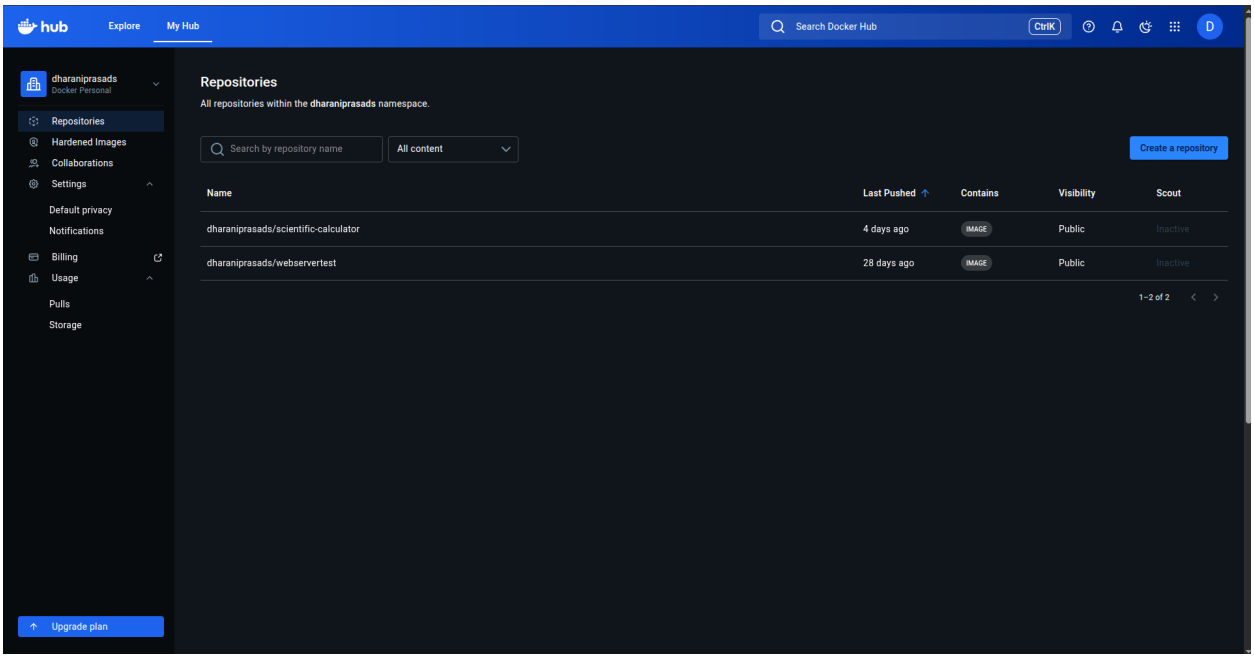


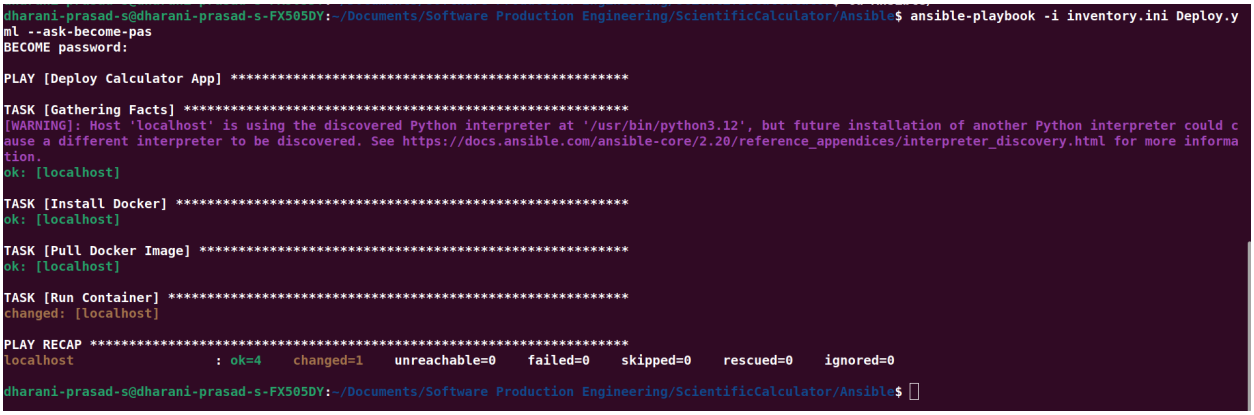
Figure 6.1: Docker Hub Repository

Chapter 7

Deployment using Ansible

Ansible is used to automate the deployment process. It pulls the Docker image from Docker Hub and runs the container on the target machine.

7.1 Ansible Playbook Execution



```
dharani-prasad-s@dharani-prasad-s-FX505DY:~/Documents/Software Production Engineering/ScientificCalculator/Ansible$ ansible-playbook -i inventory.ini Deploy.y
ml --ask-become-pas
BECOME password:

PLAY [Deploy Calculator App] ***************************************************

TASK [Gathering Facts] *********************************************************
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could c
ause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more informa
tion.
ok: [localhost]

TASK [Install Docker] **********************************************************
ok: [localhost]

TASK [Pull Docker Image] *******************************************************
ok: [localhost]

TASK [Run Container] ***********************************************************
changed: [localhost]

PLAY RECAP *********************************************************************
localhost                  : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

dharani-prasad-s@dharani-prasad-s-FX505DY:~/Documents/Software Production Engineering/ScientificCalculator/Ansible$
```

Figure 7.1: Ansible Playbook Deployment Output

7.2 Ansible Command Used

```
ansible-playbook -i inventory.ini Deploy.yml --ask-become-pass
```

Chapter 8

Scientific Calculator Application Output

After deployment, the Docker container runs the Scientific Calculator application which allows users to perform mathematical operations.

- Square Root
- Factorial
- Natural Log
- Power

8.1 Application Execution

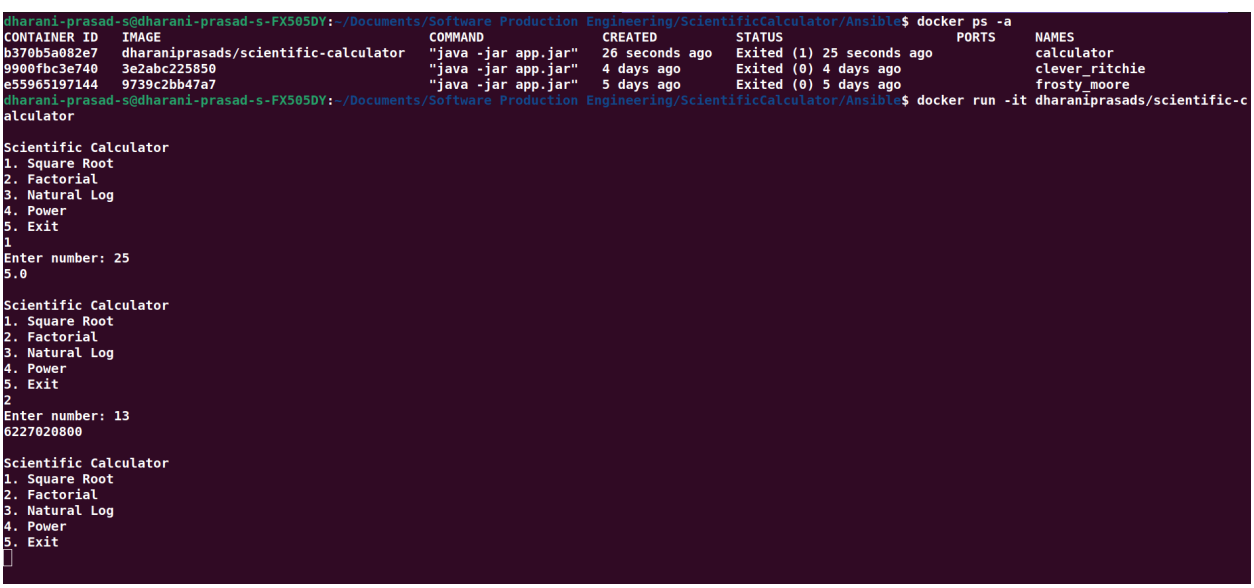


Figure 8.1: Scientific Calculator Running in Docker Container

Chapter 9

Project Links

GitHub Repository

https://github.com/dharani070707/ScientificCalculatorMiniProject

Docker Hub Repository

https://hub.docker.com/r/dharaniprasads/scientific-calculator

Chapter 10

Conclusion

This project successfully demonstrates the implementation of a DevOps pipeline for a Scientific Calculator application.

By integrating tools such as GitHub, Jenkins, Docker, Docker Hub, and Ansible, the entire process of building, testing, packaging, and deploying the application is automated.

The pipeline ensures faster development cycles, reliable deployment, and efficient management of software applications.